Midterm Mastery Report Part II Step III

Wilson Neira

Step III Downstream dependency meltdown -> circuit breaker + bulkhead + fail-fast

Story: /products/search depends on a fake “downstream service” (simulating a model service / DB / external API).

Bad version: unlimited concurrent downstream calls + no timeout -> requests pile up, CPU spikes, latency grows, and you can even force task crashes/restarts (like an outage).

Fixed version: bulkhead caps concurrent downstream work, fail-fast timeouts prevent request pileups, and a circuit breaker stops hammering the downstream when it’s failing.

Prevents cascading failure and keeps the service responsive under partial “outages”.

A) Problem Deployment (crashy/fails)

1. To HW6 main.go add:
    - /downstream = intentionally slow/flaky dependency
    - /products/search = calls downstream with No bulkhead + No timeout (bad)
    - “crash mode” to simulate a container crash under overload (shows restart behavior)

```go
// ---------- Downstream simulation ----------
func downstreamHandler(w http.ResponseWriter, r *http.Request) {
    // Simulate a dependency that sometimes gets slow or fails
    // 20% chance: fail
    // 30% chance: slow (250-600ms)
    // else: normal (10-40ms)
    p := rand.Float64()
    if p < 0.20 {
        http.Error(w, "downstream error", http.StatusServiceUnavailable)
        return
    }
    if p < 0.50 {
        time.Sleep(time.Duration(250+rand.Intn(350)) * time.Millisecond)
    } else {
        time.Sleep(time.Duration(10+rand.Intn(30)) * time.Millisecond)
    }
    w.WriteHeader(http.StatusOK)
    _, _ = w.Write([]byte(`{"ok": true}`))
}

// ---------- Search ----------
type SearchResponse struct {
    Products    []Product `json:"products"`
    TotalFound  int       `json:"total_found"`
    Checked     int       `json:"checked"`
    SearchTime  string    `json:"search_time,omitempty"`
    Downstream  string    `json:"downstream,omitempty"`
    Mode        string    `json:"mode,omitempty"`
}
```

```go
func searchHandler_BAD(w http.ResponseWriter, r *http.Request) {
    start := time.Now()
    q := strings.ToLower(strings.TrimSpace(r.URL.Query().Get("q")))
    mode := r.URL.Query().Get("mode") // e.g., mode=crash

    // Track concurrency; optionally crash if we overload (shows task res
    cur := atomic.AddInt32(&activeRequests, 1)
    defer atomic.AddInt32(&activeRequests, -1)

    if mode == "crash" && cur >= 200 {
        // Simulated crash: like a fatal error/OOM/panic in production
        panic("simulated crash: too many concurrent requests")
    }

    // BAD: Call downstream with no timeout and no concurrency limit.
    // Under load, goroutines pile up waiting on downstream => latency/CP
    dsURL := "http://127.0.0.1:8080/downstream"
    resp, err := http.Get(dsURL)
    downstreamStatus := "ok"
    if err != nil {
        downstreamStatus = "network_error"
```

```go
func main() {
    rand.Seed(time.Now().UnixNano())
    generateProducts(100000)

    mux := http.NewServeMux()
    mux.HandleFunc("/health", healthHandler)
    mux.HandleFunc("/downstream", downstreamHandler)
    mux.HandleFunc("/products/search", searchHandler_BAD)

    srv := &http.Server{
        Addr:    ":8080",
        Handler: mux,
    }

    log.Println("listening on :8080")
    log.Fatal(srv.ListenAndServe())
}
```

2. aws configure
3. terraform init

Start Docker Desktop

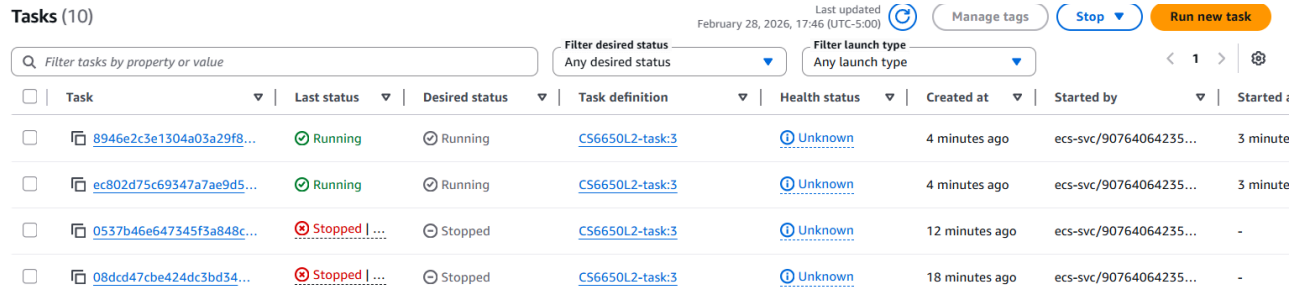
terraform apply -auto-approve `
  -replace="docker_image.app" `
  -replace="docker_registry_image.app" `

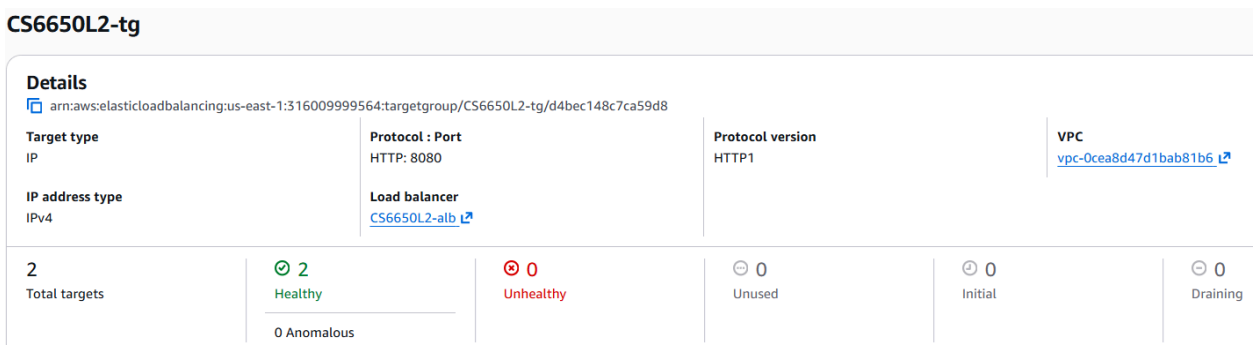
-replace="module.ecs.aws_ecs_task_definition.this" `
-replace="module.ecs.aws_ecs_service.this"

```
docker_registry_image.app: Creating...
docker_registry_image.app: Creation complete after 4s [id=sha256:e4cd3(

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

alb_dns_name = "CS6650L2-alb-1247811259.us-east-1.elb.amazonaws.com"
ecs_cluster_name = "CS6650L2-cluster"
ecs_service_name = "CS6650L2"
```

alb_dns_name = "CS6650L2-alb-1247811259.us-east-1.elb.amazonaws.com"

4. Start to prove the problem with load test





terraform apply -auto-approve -var="app_mode=bad"

locust -f locustfile_fast.py --host "http://CS6650L2-alb-1247811259.us-east-1.elb.amazonaws.com"

terraform apply -auto-approve `
-replace="docker_image.app" `
-replace="docker_registry_image.app" `
-replace="module.ecs.aws_ecs_task_definition.this" `
-replace="module.ecs.aws_ecs_service.this"

curl "http://CS6650L2-alb-1247811259.us-east-1.elb.amazonaws.com/version"

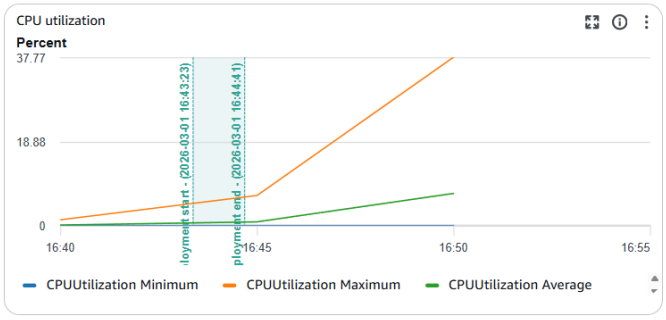
curl "http://CS6650L2-alb-1247811259.us-east-1.elb.amazonaws.com/products?q=books"

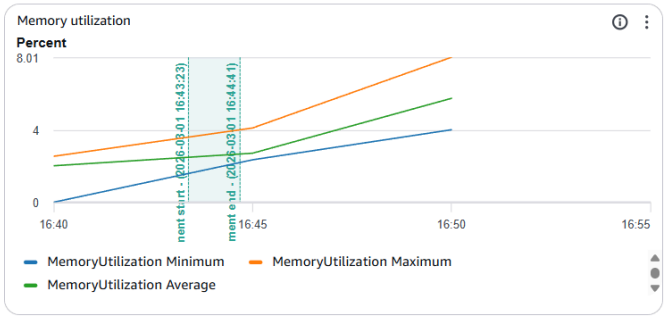
5. 20 users, 2 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	372	0	6026.83	6100	6100	6036.49	6027	6260	1475.45	3.4	0
	Aggregated	372	0	6026.83	6100	6100	6036.49	6027	6260	1475.45	3.4	0

Health

Alarm recommendations | Investigate with AI - *new* | 1h 3h 12h 1d 3d 1w Custom (15m) | UTC timezone | Explore related



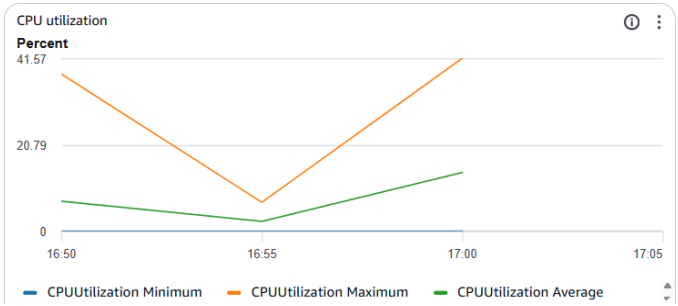


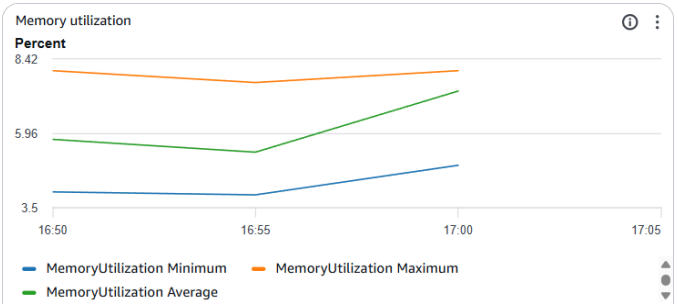
6. 200 users, 2 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	3720	0	6021.61	6100	6100	6039.18	6022	6177	1415.92	34	0
	Aggregated	3720	0	6021.61	6100	6100	6039.18	6022	6177	1415.92	34	0

Health

Alarm recommendations | Investigate with AI - *new* | 1h 3h 12h 1d 3d 1w Custom (15m) | UTC timezone | Explore related



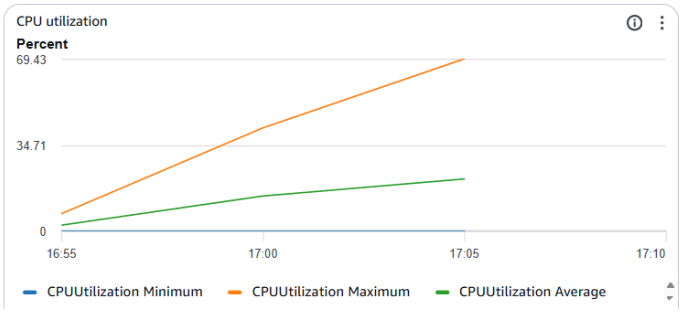


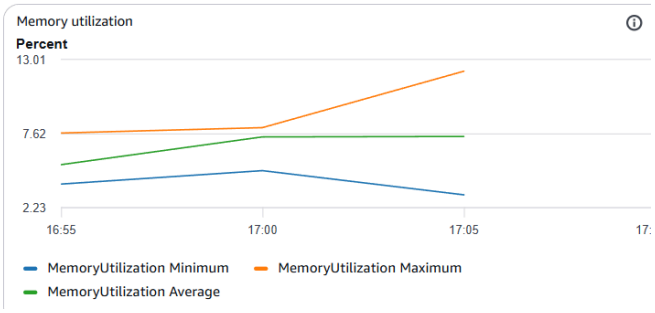
7. 400 users, 2 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	7440	0	6019.9	6100	6100	6044.05	6020	6190	1433.24	69.4	0
	Aggregated	7440	0	6019.9	6100	6100	6044.05	6020	6190	1433.24	69.4	0

Health

Alarm recommendations | Investigate with AI - *new* | 1h 3h 12h 1d 3d 1w Custom (15m) | UTC timezone | Explore related





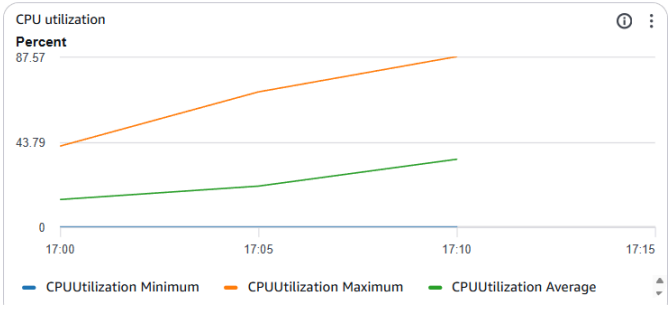
8. 1000 users, 2 minutes

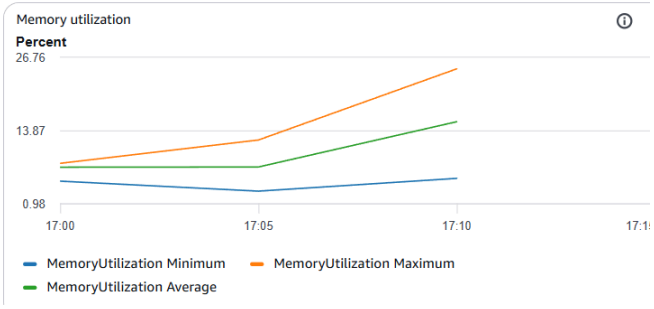
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	15976	0	6600	7600	8300	6707.02	6028	9484	1437.65	154	0
	Aggregated	15976	0	6600	7600	8300	6707.02	6028	9484	1437.65	154	0

# Failures	Method	Name	Message
97348	GET	/products/search	LocustBadStatusCode(code=502)

Health

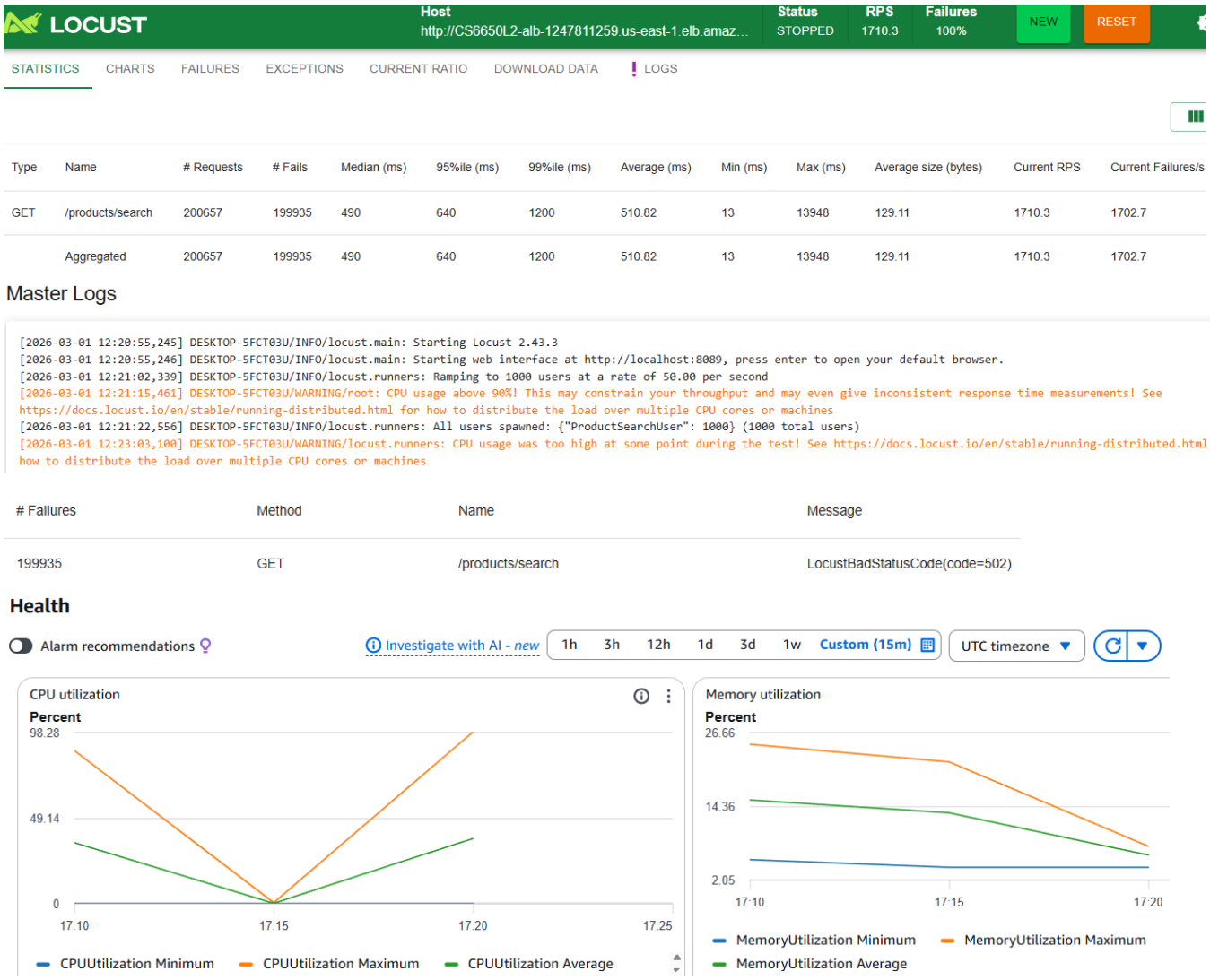
Alarm recommendations | Investigate with AI - *new* | 1h 3h 12h 1d 3d 1w Custom (15m) | UTC timezone | Explore related





10. 1000 users, 2 minutes, crash mode

locust -f locustfile_fast_crash.py --host “http://CS6650L2-alb-1247811259.us-east-1.elb.amazonaws.com”



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	200657	199935	490	640	1200	510.82	13	13948	129.11	1710.3	1702.7
	Aggregated	200657	199935	490	640	1200	510.82	13	13948	129.11	1710.3	1702.7

# Failures	Method	Name	Message
199935	GET	/products/search	LocustBadStatusCode(code=502)

B) Fix Deployment (circuit breaker + bulkhead + fail-fast)

1. Fix the Search Handler

locust -f locustfile_fast.py --host “http://CS6650L2-alb-1247811259.us-east-1.elb.amazonaws.com”

terraform apply -auto-approve -var="app_mode=fixed"

```go
func searchHandler_FIXED(w http.ResponseWriter, r *http.Request) {
    start := time.Now()
    q := strings.ToLower(strings.TrimSpace(r.URL.Query().Get("q")))

    downstreamStatus, _ := callDownstreamWithProtections()
    // Even if downstream fails, we still respond quickly (graceful degra

    checked := 0
    results := make([]Product, 0, 20)
    totalFound := 0

    for i := 0; i < len(products) && checked < 100; i++ {
        checked++
        p := products[i]
        if q == "" {
            continue
        }
        if strings.Contains(strings.ToLower(p.Name), q) || strings.Contai
            totalFound++
            if len(results) < 20 {
```

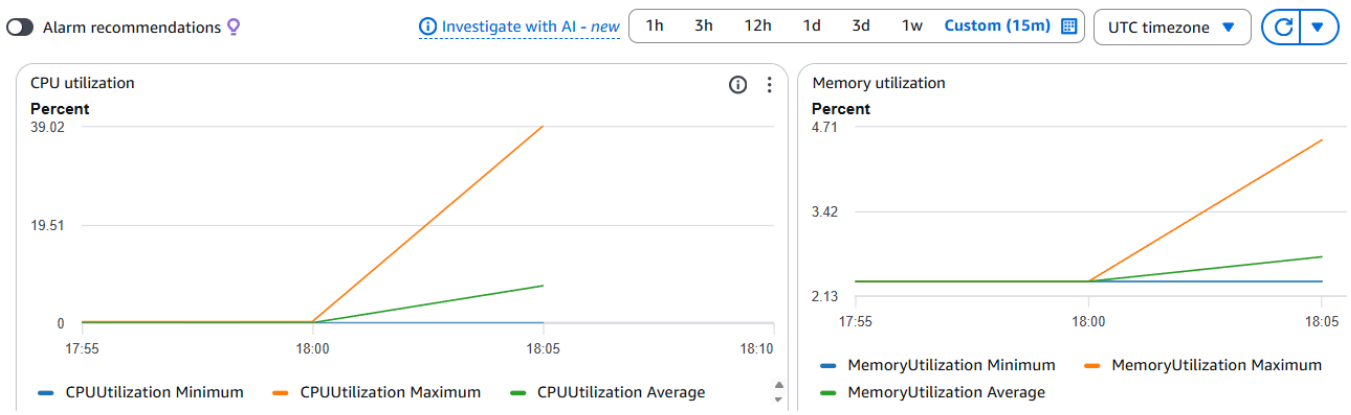
```go
func main() {
    rand.Seed(time.Now().UnixNano())
    generateProducts(100000)

    mux := http.NewServeMux()
    mux.HandleFunc("/health", healthHandler)
    mux.HandleFunc("/downstream", downstreamHandler)
    mux.HandleFunc("/products/search", searchHandler_FIXED)

    srv := &http.Server{
        Addr:    ":8080",
        Handler: mux,
    }

    log.Println("listening on :8080")
    log.Fatal(srv.ListenAndServe())
}
```

Deploy Fixed Version with terraform taint docker_image.app and terraform apply -auto-approve
terraform apply -auto-approve -var="app_mode=fixed"

2. 20 users, 2 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	110528	0	19	26	33	19.89	9	331	1430.22	945.9	0
	Aggregated	110528	0	19	26	33	19.89	9	331	1430.22	945.9	0

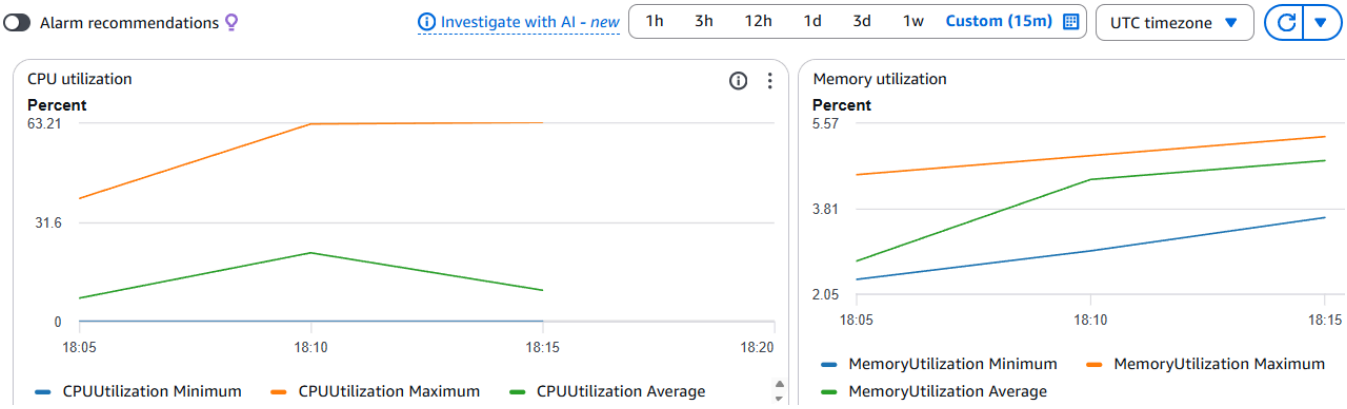
Health



3. 200 users, 2 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	249129	0	63	98	130	63.88	11	532	1431.43	2180.4	0
	Aggregated	249129	0	63	98	130	63.88	11	532	1431.43	2180.4	0

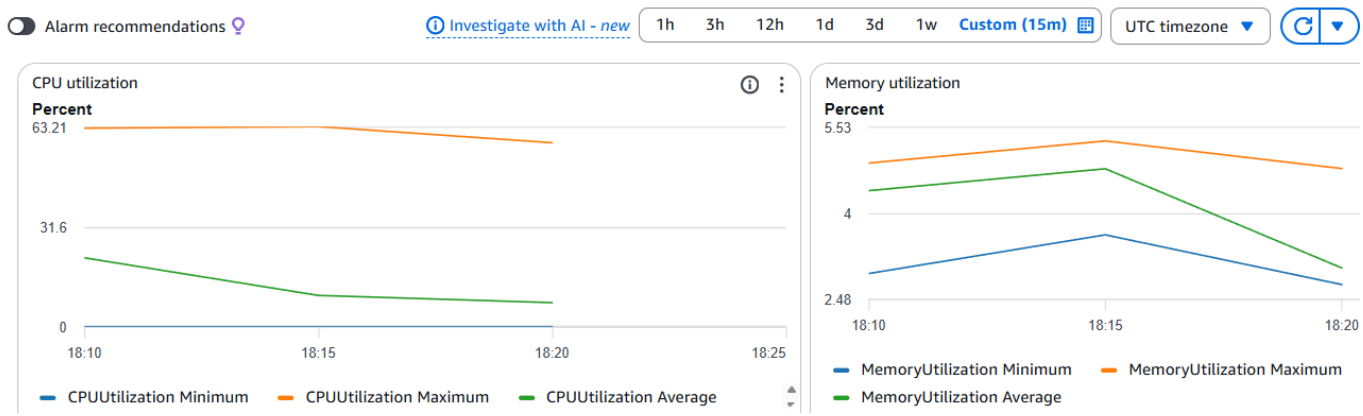
Health



4. 400 users, 2 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	243724	0	160	210	260	159.15	12	1293	1433.5	2085	0
	Aggregated	243724	0	160	210	260	159.15	12	1293	1433.5	2085	0

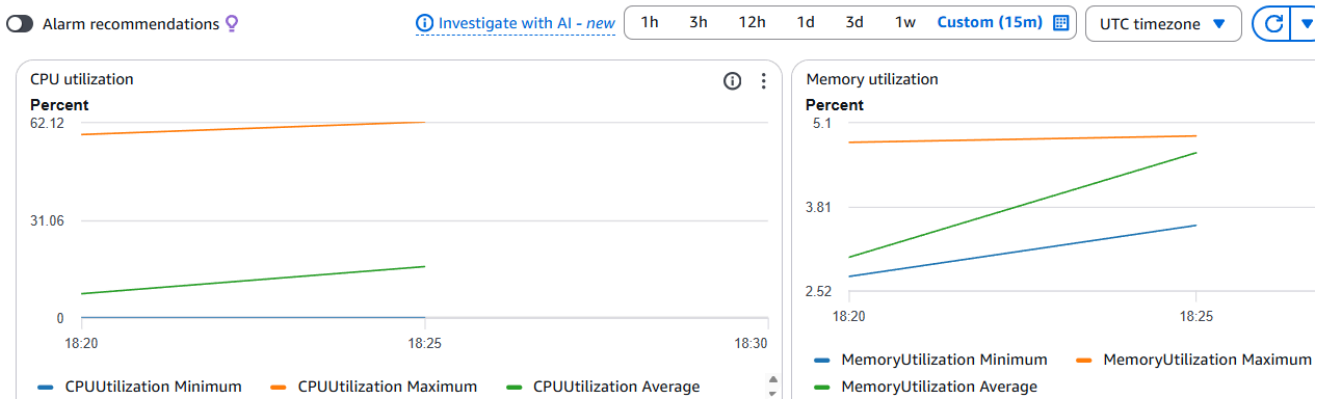
Health



5. 1000 users, 2 minutes

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	208558	0	470	830	1100	492.24	14	14955	1432.31	1777.9	0
	Aggregated	208558	0	470	830	1100	492.24	14	14955	1432.31	1777.9	0

Health



6. 1000 users, 2 minutes, crash mode

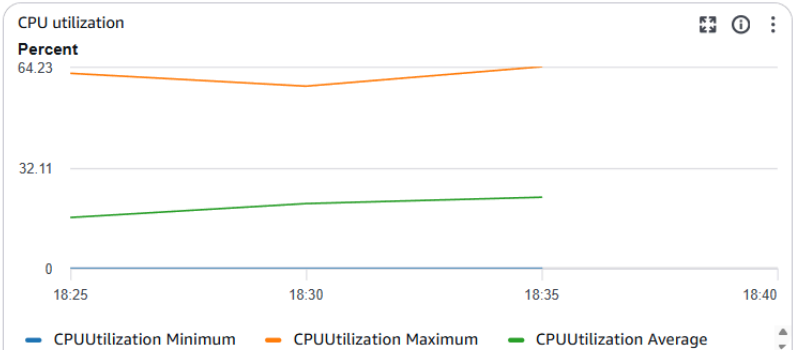
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search	223828	0	460	670	910	459.13	13	13209	2091.95	1992.4	0
	Aggregated	223828	0	460	670	910	459.13	13	13209	2091.95	1992.4	0

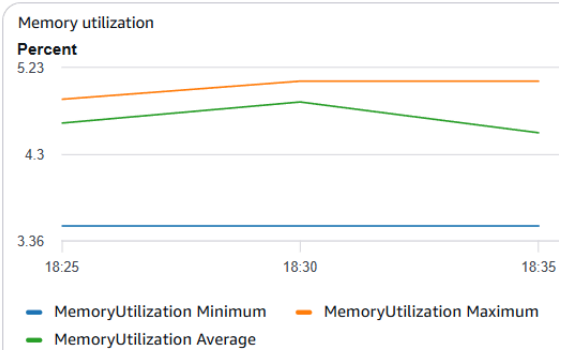
Master Logs

```
[2026-03-01 13:36:29,651] DESKTOP-5FCT03U/INFO/locust.main: Starting Locust 2.43.3
[2026-03-01 13:36:29,652] DESKTOP-5FCT03U/INFO/locust.main: Starting web interface at http://localhost:8089, press enter to open your default browser.
[2026-03-01 13:36:39,627] DESKTOP-5FCT03U/INFO/locust.runners: Ramping to 1000 users at a rate of 50.00 per second
[2026-03-01 13:36:49,954] DESKTOP-5FCT03U/WARNING/root: CPU usage above 90%! This may constrain your throughput and may even give inconsistent response time measurements! See
https://docs.locust.io/en/stable/running-distributed.html for how to distribute the load over multiple CPU cores or machines
[2026-03-01 13:36:59,872] DESKTOP-5FCT03U/INFO/locust.runners: All users spawned: {"ProductSearchUser": 1000} (1000 total users)
[2026-03-01 13:38:40,559] DESKTOP-5FCT03U/WARNING/locust.runners: CPU usage was too high at some point during the test! See https://docs.locust.io/en/stable/running-distributed.html for
how to distribute the load over multiple CPU cores or machines
```

Health

Alarm recommendations | Investigate with AI - *new* | 1h | 3h | 12h | 1d | 3d | 1w | Custom (15m) | UTC timezone





7. terraform destroy -auto-approve